

Week 1: Data Preparation – Robust Data Augmentation

Objective

The objective of this phase was to prepare the image data for training by applying **robust data augmentation techniques**. Since PCB images can vary due to lighting conditions, camera angle, and positioning in a real manufacturing environment, data augmentation helps the model generalize better and avoid overfitting.

Approach

To simulate real-world variations, **Keras ImageDataGenerator** was used to perform **real-time data augmentation during training**. Instead of creating augmented images and saving them to disk, transformations are applied dynamically while feeding data to the model.

The following augmentation techniques were applied:

- **Rotation** ($\pm 20^\circ$) to handle camera angle variations
- **Width and height shifting** (10%) to simulate misalignment
- **Zooming** (up to 15%) to account for distance variations
- **Horizontal flipping** to improve orientation robustness
- **Brightness variation** to simulate lighting changes
- **Nearest-neighbor filling** to handle empty pixels after transformation

Implementation

A training image generator was created using `ImageDataGenerator` with the above augmentation parameters. A separate validation generator was created **without augmentation** to ensure unbiased evaluation.

To verify the correctness of augmentation, multiple augmented images from a single batch were visualized using Matplotlib.

Visualization and Verification

Augmented image batches were visualized to ensure that:

- Defect patterns remain visible
- Augmentations are realistic
- Images are sufficiently diverse
- No distortion that could confuse the model is introduced

The visualization confirmed that the generated images closely resemble real PCB variations while maintaining structural integrity.

Outcome

- The model receives **diverse and realistic training samples**
- Overfitting risk is reduced
- The model becomes robust to real-world PCB imaging conditions
- Augmentation pipeline is validated visually before training

This completed the **data preparation phase** and ensured the dataset was suitable for training a high-accuracy defect detection model.

Week 2: Core Modeling – Transfer Learning for Accurate Defect Detection

Objective

The objective of this phase was to build an accurate and efficient deep learning model for PCB defect classification by using **transfer learning**. Instead of training a CNN from scratch, a **pre-trained model** was used to improve accuracy, reduce training time, and handle limited dataset size effectively.

Approach

A **MobileNetV2** architecture pre-trained on the ImageNet dataset was selected due to its balance between **high accuracy and computational efficiency**, making it suitable for real-time quality control systems.

The modeling strategy consisted of two main steps:

1. **Freezing the base model** to preserve learned low-level and mid-level features such as edges, patterns, and textures.
2. **Adding a custom classification head** tailored for binary classification (Pass / Defect).

Model Architecture

- **Base Model:** MobileNetV2 (pre-trained, `include_top=False`)
- **Feature Aggregation:** Global Average Pooling
- **Dense Layer:** Fully connected layer with ReLU activation
- **Regularization:** Dropout layer to prevent overfitting
- **Output Layer:** Sigmoid activation for binary classification

Only the custom classification layers were trained initially, ensuring stable and efficient learning.

Training Strategy

- **Loss Function:** Binary Cross-Entropy

- **Optimizer:** Adam with a low learning rate
- **Evaluation Metric:** Accuracy
- **Data:** Augmented training data and non-augmented validation data

Training progress was monitored using **learning curves (training vs validation loss and accuracy)** to detect overfitting.

Overfitting Control

To prevent overfitting:

- Data augmentation was applied only to training data
- Dropout was added in the classification head
- Validation performance was closely monitored

A stable decrease in validation loss confirmed good generalization.

Outcome

- The model achieved **low validation loss**
- No significant overfitting was observed
- Training was faster compared to training from scratch
- The model is suitable for **real-time PCB quality inspection**

This completed the **core modeling phase**, producing a robust and production-ready defect classification model.

Week 3: Interpretability (Grad-CAM)

Objective

To improve model transparency and trust by visually explaining how the CNN makes defect predictions on PCB images.

Approach

Grad-CAM (Gradient-weighted Class Activation Mapping) was implemented on the trained MobileNetV2 model. Gradients of the predicted class were used to generate heatmaps highlighting the most influential regions in the image.

Implementation

- Extracted feature maps from the final convolutional layer
- Computed gradients of predicted class scores
- Generated and overlaid heatmaps on original PCB images

Results

The Grad-CAM visualizations consistently highlighted **actual defect-prone regions**, such as solder joints and circuit tracks, rather than irrelevant background areas. This confirms that the model learned meaningful visual patterns.

Conclusion

Grad-CAM significantly improves the interpretability of the defect detection system, making it suitable for real-world industrial deployment where explainability is critical.

Week 4: Inference Optimization

Objective

The objective of this task is to enable **fast and efficient model inference** for PCB defect detection in a production-like environment. The trained MobileNetV2 model must load quickly and perform accurate predictions on incoming PCB images.

Model Optimization and Loading

The trained CNN model was saved in an optimized **.h5 format** to reduce inference overhead. This optimized model is directly loaded using TensorFlow, allowing the system to perform predictions without retraining or recompilation.

Image Preprocessing

To ensure consistency with training, each PCB image is:

- Resized to **224 × 224**
- Normalized to pixel values between **0 and 1**
- Converted into a batch format suitable for model input

This preprocessing ensures reliable and stable predictions during inference.

Inference and Visualization

Single PCB images are processed using OpenCV. The model predicts the probability of a defect, which is classified as:

- **Defect** if confidence > 0.5
- **Normal** otherwise

The predicted label and confidence score are overlaid on the image. Since the system runs in Google Colab, results are displayed using `cv2.imshow()` for safe visualization.

Conclusion

This inference pipeline demonstrates a **production-ready PCB inspection system**. By combining an optimized model, efficient preprocessing, and fast prediction with visual feedback, the system meets real-time inspection requirements and can be easily extended to live camera-based deployment.